

DOCUMENTACIÓN Y ANÁLISIS DEL PROYECTO

Fundamentos de programación en JavaScript

Proyecto:

Plataforma de Gestión e Interacción Académica Estudiantil

1. Descripción General del Sistema

El proyecto desarrollado consiste en un sistema automatizado en JavaScript, diseñado para optimizar y agilizar los procesos de administración escolar en el aula. Permite almacenar datos en memoria de sesión, edición dinámica de notas, visualización de diagnóstico sobre rendimiento académico, en un menú de control continuo guiado por el usuario.

2. Documentación Técnica de Componentes

A. Estructuras de Almacenamiento de Datos

- **listaAlumnos (Arreglo Global):** Funciona como la base de datos central en memoria. Almacena dinámicamente cada objeto de alumno registrado durante la sesión.
- **Objeto nuevoAlumno:** Estructura de datos compleja que encapsula propiedades primitivas de tipo texto (**nombre**) y una colección lineal ordenada (**notas**) de tres elementos numéricos.

B. Métodos Internos del Objeto

El sistema implementa métodos, dotando al objeto de autonomía lógica mediante propiedades de función utilizando el puntero **this**:

- **obtenerEstado(promedio):** Evalúa cualitativamente el desempeño. Retorna la cadena "APROBADO" si el valor es mayor o igual a 40, o "REPROBADO" en caso contrario.
- **actualizarNota(numNota, nuevaNota):** Accede de forma directa al arreglo de notas del propio objeto y altera el valor en el índice correspondiente (**numNota - 1**), permitiendo la edición en tiempo real de las notas.

C. Funciones Globales Operacionales

- **agregarAlumno()**: Función interactiva encargada de capturar los datos del usuario a través de `prompt()`, validar la integridad de la información e instanciar el objeto estudiante.
- **calcularPromedio(arregloNotas)**: Algoritmo matemático puro y reutilizable que recorre cualquier arreglo numérico para acumular los valores y retornar el promedio aritmético redondeado a un decimal mediante `.toFixed(1)`.
- **modificarNotasAlumno()**: Gestiona el motor de búsqueda en la colección global y coordina la ejecución del método interno de reescritura de notas del objeto.
- **mostrarReporteGeneral()**: Modula la salida de datos imprimiendo masivamente en la consola (`console.log`) la ficha académica completa del curso.

3. Análisis de Arquitectura y Buenas Prácticas

A. Blindaje y Validación de Datos contra Errores

Para asegurar la estabilidad del flujo y evitar corrupciones en los cálculos, el sistema implementa **bucles centinela** (`while(true)`) **combinados con condicionales estrictos** e `isNaN()`. Esto obliga al usuario a ingresar exclusivamente valores dentro de la escala numérica válida (10 a 70). Si el usuario comete un error, el programa no se cae; emite una alerta educativa y solicita el dato nuevamente.

B. Recorridos de Datos Modernos

En lugar de depender exclusivamente de bucles indexados tradicionales (`for`), las operaciones masivas de búsqueda y generación de reportes implementan el método de orden superior `.forEach(function(alumno){ ... })`. Esto elimina la necesidad de gestionar manualmente contadores de índices intermedios, aumentando la legibilidad del código y adoptando los estándares modernos de desarrollo de JavaScript (ES6+).

C. Control de Flujo Continuo e Indestructible

El bucle principal `while (opcion !== "5")` y su estructura selectiva `switch` garantizan que la aplicación permanezca activa tras concluir cualquier tarea. Adicionalmente, el sistema cuenta con controles de excepción tempranos: si el usuario cancela la búsqueda de un alumno o ejecuta el reporte general sin haber inscrito estudiantes previamente, el software lo detecta de inmediato, emite un aviso controlado mediante `alert()` y utiliza la sentencia `return` para regresar al menú principal de forma segura, evitando cierres involuntarios del sistema.